Laboratory Work 2

Algorithmic Thinking, Pseudocode and Flowcharts

Problem solving · Algorithms · Pseudocode · Flowcharts · Trace tables

Week: 2 — second session
Duration: 2 hours 15 minutes
Format: guided theory, then individual practice
Tools: pen, paper and a ruler for flowcharts
Prerequisite: Lab 1 (Number Systems) — not required, but Section G reuses its repeated-division idea

Name:

Group:

Date:

Learning outcomes

By the end of this lab you should be able to:

- **LO1.** Describe the five stages of the program development cycle, and say which stage produces an algorithm and which produces code.
- **LO2.** State the properties every algorithm must have, and write a correct step-by-step algorithm for a stated problem.
- **LO3.** Express an algorithm as readable pseudocode using assignment, selection and iteration.
- **LO4.** Draw a flowchart with the standard symbols, including labelled decision branches and a loop.
- **LO5.** Trace an algorithm with a trace table and use it to find a logic error.
- **LO6.** Recognise the common logic faults: infinite loops, uninitialised variables, unreachable branches and mis-ordered conditions.

This lab covers the first half of Lecture 2 (*Algorithmic Thinking and Solution Design*). Lab 1 covered the second half — number representation. Together they complete Week 2.

How this lab is organised

Work through the document in order. **Part I** explains each idea and gives a worked example; after most ideas there is a short **Quick check** that you solve immediately, before moving on. **Part II** is the graded practice set. Do not start Part II until your instructor says so.

Block	Content	Time
–	Introduction and learning outcomes	5 min
PART I — THEORY AND WORKED EXAMPLES		40 min
§1	From problem to program: the development cycle	5 min
§2	What an algorithm is, and what it must guarantee	6 min
§3	Pseudocode: conventions and assignment	6 min
§4	Selection: `IF`, `ELSE`, chained conditions	5 min
§5	Iteration: `WHILE`, `FOR`, accumulators	6 min
§6	Flowcharts: symbols, decisions and loops	7 min
§7–§8	Trace tables, and the faults they expose	5 min
–	*Short break*	5 min
PART II — PRACTICE		80 min
A–B	Warm-up and step-by-step algorithms	14 min
C–D	Pseudocode: selection, then iteration	20 min
E–F	Flowchart symbols and three flowcharts	18 min
G	Trace tables	10 min
H	Find the mistake	6 min
I–J	True/false and multiple choice	8 min
K	Looking ahead: pseudocode → C++	4 min
L	Bonus challenges *(not timed)*	—
–	Submission checklist and wrap-up	5 min
Total		**2 h 15 min**

Pseudocode conventions used in this lab. Keywords in capitals: `INPUT`, `SET`, `PRINT`, `IF . . . THEN`, `ELSE`, `WHILE . . . DO`, `FOR`. Assignment is written `SET x = 5` or $x \leftarrow 5$ — both mean *store 5 in x*. Indentation shows what belongs inside an `IF` or a loop, so indent carefully: in pseudocode the indentation *is* the structure.

PART I — Theory and worked examples

1 From problem to program

Programming is not typing. A program is a *solution to a problem*, and most of the thinking happens before a single line of code is written. Every program you will write in CS102 goes through the same five stages.

	Stage	What you produce
1	**Analyse the problem**	A clear statement of the *input*, the *output*, and the process that turns one into the other.
2	**Plan the solution**	The **algorithm** — as pseudocode, a flowchart, or both. You check it with a trace table and realistic test data.
3	**Code the program**	Source code in a real language, following that language's **syntax**. A translator (compiler) turns it into machine code.
4	**Test**	Runs on data chosen to expose faults. Failures send you back to stage 2, not stage 3.
5	**Deploy and maintain**	Documentation, user manuals, and updates as requirements change.

Algorithm versus syntax

These are the two halves of a working program, and they fail in different ways.

- The **algorithm** is the step-by-step logic that reaches the goal. It is language-independent — the same algorithm can be written in C++, Python or on paper.
- The **syntax** is the set of rules of a particular language that dictate how statements must be written.

A **syntax error** is caught by the compiler and is usually cheap to fix. A **logic error** compiles perfectly and produces the wrong answer — the compiler cannot help you, because you asked exactly what you wrote. This lab is entirely about avoiding the second kind.

Quick check 1. A program compiles without any error, runs, and prints an average of 47 when the correct answer is 74. Which stage of the cycle do you return to, and why?

2 What an algorithm is

Algorithm

An **algorithm** is a finite, ordered sequence of unambiguous steps that solves a problem or performs a task.

Four words in that definition are doing real work:

Property	What it demands
Finite	It must stop. An algorithm that loops forever has not solved anything.
Ordered	The steps run in a definite sequence. Swapping two steps may change the result.
Unambiguous	Every step means exactly one thing. "Add some salt" is not a step; "add 5 grams of salt" is.
Effective	Each step is simple enough to actually be carried out. "Guess the answer" is not effective.

An algorithm takes *zero or more* inputs and produces *one or more* outputs. Zero inputs is legal — an algorithm that prints the numbers 1 to 10 needs nothing from the user.

Decomposition: the al-Khwārizmī principle

The word *algorithm* comes from al-Khwārizmī, the 9th-century mathematician. The principle that carries his name is the single most useful habit in programming:

Any complex problem can be broken into simpler sub-problems. Solve each sub-problem in order; when all of them are solved, the original problem is solved too.

Worked example 2.1 — average of three numbers

Problem: read three numbers and print their average.

Analyse. Input: N_1, N_2, N_3. Output: their average. Process: add the three, divide by 3.

Algorithm.

1. Start.
2. Get values for N_1, N_2 and N_3.
3. Set $sum \leftarrow N_1 + N_2 + N_3$.
4. Set $average \leftarrow sum/3$.
5. Print $average$.
6. Stop.

Note that steps 3 and 4 could be merged into one, and that the order matters: you cannot divide before you have added.

An algorithm is only as good as its least precise step. "Cook the rice until it is ready" is how a person writes a recipe; a computer needs "simmer for 20 minutes". When you catch yourself writing *some*, *a few*, *until it looks right* or *etc.*, you have found a step that is not yet an algorithm.

Quick check 2. Which of these four steps are *not* acceptable in an algorithm, and why?

(a) Set $count \leftarrow count + 1$.
(b) Repeat until the user gets bored.
(c) If $x > 10$, print "large".
(d) Sort the list somehow.

3 Pseudocode

Pseudocode

Pseudocode expresses an algorithm using English-like phrases arranged to look like statements in a programming language — but with *no strict syntax rules*. It is a compromise: more precise than prose, less fussy than real code, and readable by anyone regardless of the language they program in.

Because there is no compiler, there is no single correct spelling. All four of these lines mean the same thing, and all four are acceptable:

`Set the value of A to (B+C)` | $A \leftarrow B + C$ | `A = B + C` | `SET A = B + C`

This lab uses the `SET` form. What matters is that you are *consistent* within one piece of pseudocode, and that a reader can tell exactly what each line does.

Algorithm or pseudocode?

Students often ask what the difference is. They describe the same logic at different levels of detail.

	Algorithm	**Pseudocode**
Written in	plain language or numbered logic	a programming-like structure, still not real code
Focuses on	*what* to do	*how* to express the steps precisely
Depends on a language?	no	no
Used	while planning and problem solving	just before writing actual code

Example	1. Start
	2. Read two numbers
	3. Add the numbers
	4. Display the result
	5. Stop
		`START`
		`INPUT a, b`
		`SET sum = a + b`
		`PRINT sum`
		`END`

Assignment is not equality

`SET x = x + 1` is not an equation — as mathematics it would be false for every x. Read it **right to left**: *take the current value of* `x`*, add one, and store the result back into* `x`. If `x` held 7, it now holds 8. This single line is the engine of every counter and every loop you will write.

Quick check 3. After these three lines, what does each variable hold?

```
SET a = 4   SET b = a + 3   SET a = b * 2
```

4 Selection: making a decision

`IF` runs a block only when a condition is true. `ELSE` gives the alternative.

```
INPUT score
IF score >= 60 THEN
    PRINT "Pass"
ELSE
    PRINT "Fail"
```

```
INPUT N
IF N > 0 THEN
    PRINT "Positive"
ELSE IF N < 0 THEN
    PRINT "Negative"
ELSE
    PRINT "Zero"
```

A chain of `ELSE IF` tests the conditions *in order* and stops at the first one that is true. That makes the order part of the logic, not a matter of taste.

Worked example 4.1 — why the order of conditions matters

Classify an average: `"Excellent"` if ≥ 90, `"Good"` if $75 \leq avg < 90$, otherwise `"Needs improvement"`.

Correct — strictest test first

```
INPUT avg
IF avg >= 90 THEN
    PRINT "Excellent"
ELSE IF avg >= 75 THEN
    PRINT "Good"
ELSE
    PRINT "Needs improvement"
```

Wrong — loosest test first

```
INPUT avg
IF avg >= 75 THEN
    PRINT "Good"
ELSE IF avg >= 90 THEN
    PRINT "Excellent"
ELSE
    PRINT "Needs improvement"
```

In the version on the right, $avg = 95$ prints `"Good"`: the first condition already caught it, so the `>= 90` branch can *never* run. A branch that no input can reach is called **unreachable code**, and it is a logic error the compiler will not report.

Notice also what the correct version buys you: once `>= 90` has failed, writing `avg >= 75` is enough — you do not need `avg >= 75 AND avg < 90`, because reaching that line already guarantees $avg < 90$.

> **Quick check 4.** What does the *correct* version print for $avg = 90$? For $avg = 75$? For $avg = 74.9$?

5 Iteration: repeating work

A loop repeats a block while a condition holds. `WHILE` tests *before* each pass, so a `WHILE` loop may run zero times.

```
SET i = 1
WHILE i <= 5 DO
    PRINT i
    SET i = i + 1
```

Prints `1 2 3 4 5`. Three parts must all be present: **initialise** (`i = 1`), **test** (`i <= 5`), **update** (`i = i + 1`). Drop the update and the loop never ends.

Two patterns appear in almost every loop you will ever write:

Pattern	Shape	Why it is set up that way
Counter	`SET i = 0` … `SET i = i + 1`	Counts how many times something happened.
Accumulator	`SET total = 0` … `SET total = total + x`	Builds a running sum. It starts at 0 because 0 is the sum of no numbers.

Worked example 5.1 — sum of N numbers

Read N, then read N numbers and print their sum.

```
INPUT n
SET total = 0 // accumulator: sum of nothing is 0
SET i = 1 // counter
WHILE i <= n DO
    INPUT x
    SET total = total + x
    SET i = i + 1 // without this line the loop never ends
PRINT total
```

Trace it for $n = 3$ with inputs 4, 7, 2: after pass 1 $total = 4$, $i = 2$; after pass 2 $total = 11$, $i = 3$; after pass 3 $total = 13$, $i = 4$; the test $4 \leq 3$ fails and the loop exits. It prints 13. ✓

Note that if the user enters $n = 0$, the test fails immediately, the body never runs, and the algorithm prints 0 — which is the right answer. Initialising the accumulator correctly is what makes the empty case work for free.

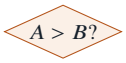
`FOR` is shorthand. When the number of passes is known in advance you may write `FOR i = 1 TO n DO`, which initialises, tests and updates `i` for you. It is exactly the `WHILE` loop above with the bookkeeping folded into one line. Use `WHILE` when the number of passes depends on the data.

Quick check 5. How many lines does this print, and what is the last one?

```
SET i = 10 WHILE i > 0 DO PRINT i SET i = i - 3
```

6 Flowcharts

A **flowchart** is the same algorithm drawn instead of written. The blocks are the operations; the arrows are the order in which they happen.

Symbol	Name	Used for
	Oval (terminator)	`Start` and `End`. Exactly one of each.
	Parallelogram	Input and output: reading a value, printing a result.
	Rectangle	A process: a calculation or an assignment.
	Diamond	A decision. **Two** outgoing arrows, labelled `Yes` and `No`.
	Arrow	The flow of control from one block to the next.

Worked example 6.1 — the same algorithm three ways

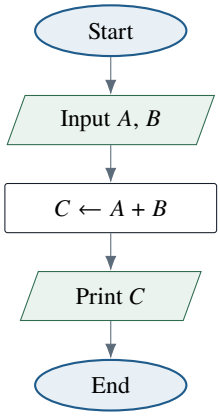
Natural language

1. Begin.
2. Input two values A, B.
3. Add the values.
4. Display the result.
5. End.

Pseudocode

```
START
INPUT A, B
SET C = A + B
PRINT C
END
```

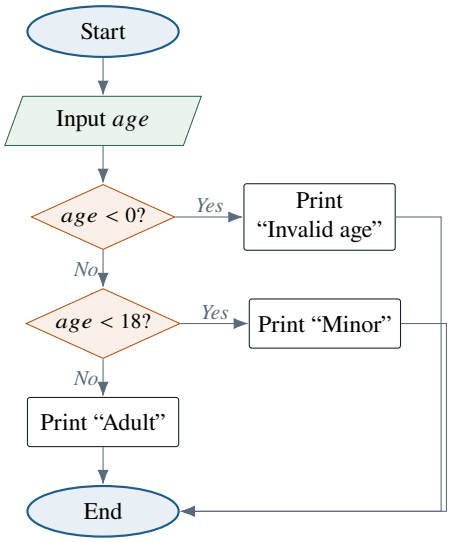
Flowchart



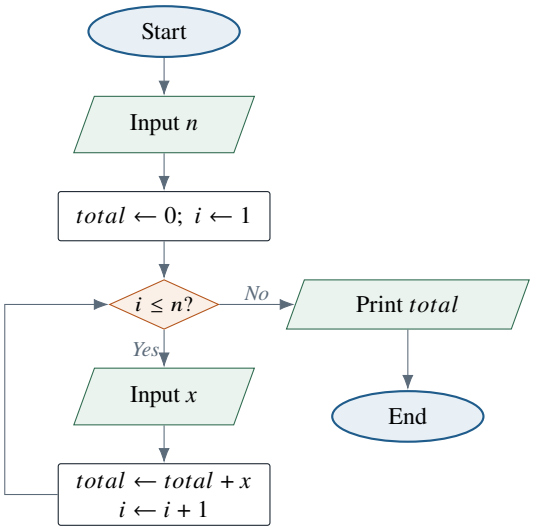
All three say exactly the same thing. Choose whichever makes the logic clearest to your reader — flowcharts win for branching and loops, pseudocode wins for long calculations.

Worked example 6.2 — a decision, and a loop

(a) Two decisions: age classification



(b) A loop: sum of N numbers



In (b) the arrow from the accumulator runs *back up* into the decision. That upward arrow is what makes it a loop — and the block it returns to must be the *test*, never the initialisation, or $total$ would be reset to 0 on every pass.

> **Three flowchart faults that cost marks.** A decision with only one outgoing arrow, or with branches that are not labelled `Yes`/`No`. A loop whose arrow returns above the initialisation. A chart with no `Start` or no `End` block.

7 Trace tables

A **trace table** records the value of every variable after every step. It is how you test an algorithm before any code exists — and how you find a logic error once code does exist.

Worked example 7.1 — tracing selection

```
SET x = 5
SET y = 7
IF x = 5 THEN
    SET y = 8
ELSE
    SET y = 0
IF y = 7 THEN
    SET x = 6
ELSE
    SET x = 3
IF x = y THEN
    SET y = 0
```

Step	Action	x	y
1	`SET x = 5`	5	—
2	`SET y = 7`	5	7
3	`x = 5`? **true** → `y = 8`	5	8
4	`y = 7`? **false** → `x = 3`	3	8
5	`x = y`? (3 = 8) **false** → no change	3	8

Final: $x = 3$, $y = 8$.

Step 4 is the one everybody gets wrong. The test `y = 7` is evaluated against the *current* value of `y`, which step 3 already changed to 8 — so it is false, and the `ELSE` branch runs. A trace table forces you to carry the new value forward instead of remembering the old one.

Tracing a loop. Give the table one row per *pass*, and add a column for the loop condition. Writing the condition down is what catches an infinite loop: if the values in the condition's column stop changing, the loop will never end.

Quick check 6. Trace this loop. What does it print?

```
SET x = 1  SET c = 0  WHILE x < 20 DO { SET x = x * 3 ; SET c = c + 1 }  PRINT x, c
```

8 Common mistakes — and how to catch them

Mistake	What to do instead
Ambiguous step	Replace every *some*, *a few*, *until it looks right* with a number or a testable condition.
Infinite loop	Check that the loop body changes a variable used in the test.
Uninitialised variable	Every counter and accumulator is set *before* the loop, never inside it.
Mis-ordered conditions	In an `ELSE IF` chain, test the strictest condition first, or later branches become unreachable.
Missing `ELSE`	Without it, the second `PRINT` runs for every input, not just the failing ones.
Boundary off by one	Decide explicitly whether the limit itself passes: $\geq$ or $>$, $\leq n$ or $< n$.
Reset inside the loop	A loop's return arrow goes back to the *test*, not to the initialisation.
Unlabelled branches	Every diamond has two arrows, each labelled `Yes` or `No`.

The habit to build: before you accept any algorithm, **trace it on paper with a boundary value** — the smallest legal input, the largest, and one just outside. An algorithm that is correct for $n = 5$ and wrong for $n = 0$ is still a broken algorithm.

PART II — Practice

> **Rules for Part II.** There is no single correct wording — pseudocode has no compiler. You are marked on whether the *logic* is correct, complete and unambiguous. Indent the body of every `IF` and every loop. Label both branches of every decision in a flowchart. Before you write an answer down, trace it once with a boundary value. Total: **100 points** plus up to 10 bonus points in Section L.

A Warm-up: make it unambiguous

[4 min] (5 points)

a) Here is an "algorithm" for making tea. Three of its six steps break one of the four properties from §2. Circle them and say which property each one breaks. (3)

	Step	Property broken (if any)
1	Boil some water.	
2	Put one teabag in a cup.	
3	Pour in 200 ml of the boiled water.	
4	Wait until it tastes right.	
5	Keep stirring.	
6	Remove the teabag and serve.	

b) Rewrite steps 1, 4 and 5 so that a machine could carry them out. (2)

B Step-by-step algorithms

[10 min] (12 points)

Write numbered steps, starting with *Start* and ending with *Stop*. Keep each step clear and unambiguous.

a) (5) **Average with validation.** Three numbers N_1, N_2, N_3 are given. Compute their average **only if all three are non-negative**. If at least one is negative, print `"Invalid input"` instead. *(6–8 steps)*

b) (3) **Comparing two numbers.** Two numbers A and B are given. Print which one is larger; if they are equal, print `"Numbers are equal"`. *(5–7 steps)*

c) (4) **Quiz classification.** A student enters three quiz scores. Compute the average and print `"Excellent"` if it is ≥ 90, `"Good"` if $75 \leq avg < 90$, and `"Needs improvement"` otherwise. *(7–9 steps)*

C Pseudocode: selection

[10 min] (10 points)

Use only `INPUT`, `SET`, `IF . . . THEN`, `ELSE` and `PRINT`. Indent the body of every `IF`.

a) (3) **Sign of a number.** A number N is given. Print `"Positive"`, `"Negative"` or `"Zero"`.

b) (2) **All equal?** Three numbers A, B, C are given. Print `"All equal"` if all three are equal, otherwise `"Not all equal"`.

c) (2) **Pass or fail.** An exam score is given. Print `"Pass"` if the score is at least 60, otherwise `"Fail"`.

d) (3) **Divisibility.** Two numbers X and Y are given. If X is divisible by Y, print `"Divisible"`; otherwise print `"Not divisible"`. Guard against $Y = 0$.

D Pseudocode: iteration

[10 min] (12 points)

Each of these needs a loop. Remember the three parts: **initialise**, **test**, **update**.

a) (3) **Sum 1 to N.** Read N and print $1 + 2 + \cdots + N$. If $N = 0$ the answer must be 0.

b) (3) **Count the positives.** Read N, then read N numbers, and print how many of them were greater than zero.

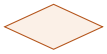
c) (3) **Largest of N.** Read N, then read N numbers, and print the largest. *Hint: what should the running maximum be before you have seen any number?*

d) (3) **Division by repeated subtraction.** Two positive integers X and Y are given. Without using $\div$ or `MOD`, print the quotient and the remainder of $X \div Y$.

E Flowchart symbols

[4 min] (5 points)

a) Name each symbol and say what it is used for. (3)

Symbol	Name	Used for
		
		
		
		

b) How many arrows leave a decision block, and how must they be labelled? (1)

c) In a loop, which block must the upward return arrow point back to, and what goes wrong if it points one block higher? (1)

Rough work — use the space below to sketch before you draw your final flowcharts in Section F.

F Flowcharts

[14 min] (18 points)

Draw a flowchart **and** write the matching pseudocode for each problem. Use a ruler. Every decision needs two labelled branches, and every chart needs exactly one `Start` and one `End`.

a) (6) **Ticket price.** A visitor's age is given.

- under 6 → print `"Free"`
- 6 to 17 → print `"Half price"`
- 18 to 64 → print `"Full price"`
- 65 and over → print `"Half price"`

Requirement: three decision blocks with labelled branches.

b) (6) **Largest of three.** Find and print the largest of three integers A, B, C.

c) (6) **Multiplication table.** Read a number N and print $N \times 1, N \times 2, \ldots, N \times 10$, one per line. **Requirement:** a loop — a decision block whose `Yes` branch eventually returns upward.

G Trace tables

[10 min] (12 points)

a) (4) **Selection.** Trace the pseudocode and fill in the table.

```
SET a = 3
SET b = 12

IF a > b THEN
    SET a = a - 2
ELSE
    SET b = b - 5

IF a + b > 9 THEN
    SET a = a + 4
ELSE
    SET b = b + 1

IF a = b THEN
    SET a = 0
```

Step	a	b	What happened
Init			Initial values assigned
1			
2			
3			
Final			Final values

b) (4) **A loop.** Trace this for $X = 20$, $Y = 6$. Add one row per pass.

```
INPUT X, Y
SET q = 0
WHILE X >= Y DO
    SET X = X - Y
    SET q = q + 1
PRINT q, X
```

Pass	$X \geq Y$?	X	q
before	—		
1			
2			
3			
4			

It prints: _____

c) (4) **Find the bug by tracing.** The author wanted this to print 1 + 2 + 3 + 4 + 5.

```
SET s = 0
SET i = 1
WHILE i < 5 DO
    SET s = s + i
    SET i = i + 1
PRINT s
```

Pass	$i < 5$?	s	i
before	—		
1			
2			
3			
4			
5			

It prints _____, but the intended answer is _____. Which **single character** must change, and to what?

H Find the mistake

[6 min] (8 points)

Each fragment below has exactly one logic fault. Name the fault and write the corrected line or lines. None of them would be caught by a compiler.

a) (2)

```
INPUT n
SET i = 1
SET s = 0
WHILE i <= n DO
    SET s = s + i
PRINT s
```

---

b) (2)

```
INPUT n
WHILE i <= n DO
    SET total = total + i
    SET i = i + 1
PRINT total
```

---

c) (2)

```
INPUT avg
IF avg >= 75 THEN
    PRINT "Good"
ELSE IF avg >= 90 THEN
    PRINT "Excellent"
ELSE
    PRINT "Needs improvement"
```

---

d) (2)

```
INPUT score
IF score >= 60 THEN
    PRINT "Pass"
PRINT "Fail"
```

I True or false

[4 min] (6 points)

Mark each statement **T** or **F**. For every **F**, write one short line saying why.

1. An algorithm must finish after a finite number of steps. _____

2. Pseudocode must follow strict syntax rules so that it can be compiled. _____

3. A flowchart and pseudocode can describe exactly the same algorithm. _____

4. A decision block in a flowchart has exactly one outgoing arrow. _____

5. An algorithm is independent of any particular programming language. _____

6. A trace table records the value of each variable after each step. _____

7. $A \leftarrow B + C$ and `SET A = B + C` mean the same thing. _____

8. Syntax errors are found during the *Plan the solution* stage. _____

9. The same problem can have more than one correct algorithm. _____

10. A `WHILE` loop always executes its body at least once. _____

11. Decomposition means breaking a complex problem into simpler sub-problems. _____

12. Every algorithm must take at least one input. _____

J Multiple choice

[4 min] (6 points)

Circle one answer per question.

1. Which is **not** required of an algorithm?
(a) it finishes (b) it is written in C++
(c) its steps are unambiguous (d) its steps are ordered

2. In a flowchart, a diamond represents
(a) a process (b) input/output
(c) a decision (d) start or end

3. A parallelogram represents
(a) a decision (b) a process
(c) start or end (d) input/output

4. Which stage produces pseudocode and flowcharts?
(a) Analyse (b) Plan (c) Code (d) Test

5. A syntax error is detected by
(a) the language translator (b) a trace table
(c) the flowchart (d) the end user

6. A trace table shows
(a) the final output only
(b) each variable's value at each step
(c) the flowchart symbols (d) compiler errors

7. `SET x = x + 1` means
(a) compare `x` with `x+1`
(b) an equation with no solution
(c) store the old `x` plus one back into `x`
(d) nothing — it is invalid

8. A `WHILE` loop whose body never changes the tested variable
(a) runs once (b) runs zero times
(c) runs forever (d) is a syntax error

9. Given `IF avg >= 90 . . . ELSE IF avg >= 75 . . .`, which value prints `"Good"`?
(a) 95 (b) 90 (c) 80 (d) 60

10. Pseudocode differs from a real language mainly because
(a) it has no strict syntax rules
(b) it runs faster
(c) it cannot express loops
(d) computers execute it directly

11. The development cycle runs in the order
(a) Code, Analyse, Plan, Test
(b) Analyse, Plan, Code, Test
(c) Plan, Analyse, Test, Code
(d) Analyse, Code, Plan, Test

12. "An algorithm must stop after a finite number of steps" is the property called
(a) definiteness (b) effectiveness
(c) finiteness (d) ordering

K Looking ahead: pseudocode → C++ [4 min] (6 points)

You will meet C++ syntax properly in Lab 3. Nothing here has to be memorised — the point is that every pseudocode construct you wrote today has a direct C++ counterpart, so the algorithm is the part that carries over.

a) Complete the table. (3)

Pseudocode	C++
`INPUT x`	`cin » x;`
`PRINT x`	
`SET x = 5`	
`IF a > b THEN`	`if (a > b) {`
`ELSE`	
`WHILE i <= n DO`	

b) Name the three things C++ demands that pseudocode does not. (2)

c) In pseudocode `IF a = b THEN` tests whether `a` and `b` are equal. What must that be written as in C++, and what does the single `=` do instead? (1)

L Bonus challenges (up to 10 points, not timed)

Attempt these only after everything above is finished. Give pseudocode for each.

1. (4) **Elevator.** The current floor and the destination floor are given. Report how many floors the elevator travels and in which direction: `"UP"`, `"DOWN"` or `"STAY"`.

2. (3) **Leap year.** A year is a leap year if it is divisible by 4, *except* that years divisible by 100 are not, *unless* they are also divisible by 400. Print `"Leap"` or `"Not leap"`.

3. (3) **Reverse the digits.** A three-digit number is given, for example 123. Print its digits reversed: 321. Use only `DIV` (integer division) and `MOD`.

Submission checklist [5 min]

Before you hand in, confirm each line. Tick the box.

- ☐ Your **name** and **group** are on the front page.
- ☐ Every algorithm in Section B starts with *Start* and ends with *Stop*.
- ☐ The body of every `IF` and every loop is **indented**.
- ☐ Every loop you wrote has all three parts: **initialise**, **test**, **update**.
- ☐ Every flowchart in Section F has one `Start`, one `End`, and **two labelled arrows** out of each diamond.
- ☐ Each trace table in Section G is filled in for **every** row, including the final row.
- ☐ Sections A, E, H and K include the short **written explanations** they ask for.
- ☐ You traced at least one answer with a **boundary value** before writing it down.

Section	Available	Earned
A Warm-up: make it unambiguous	5	
B Step-by-step algorithms	12	
C Pseudocode: selection	10	
D Pseudocode: iteration	12	
E Flowchart symbols	5	
F Flowcharts	18	
G Trace tables	12	
H Find the mistake	8	
I True or false	6	
J Multiple choice	6	
K Looking ahead: pseudocode to C++	6	
Total	**100**	
L Bonus challenges	+10	

What comes next

Everything in this lab is the plan that the rest of CS102 executes:

- **Selection** becomes `if` / `else` in Lab 4.
- **Iteration** becomes `while` and `for` in Lab 5.
- **Decomposition** becomes functions in Lab 6 — one sub-problem, one function.
- **Trace tables** become the debugger, and stay the fastest way to find a logic error.

The habit worth keeping: write the algorithm before the code. A logic error found on paper costs a minute; the same error found after a hundred lines of C++ costs an afternoon.

End of Lab 2